Managing access control in Docker

In this train, we'll explore methods for managing access to Docker containers and resources, ensuring that only authorised users and services have access.

Learning objective

> - Be able to manage access to Docker containers and resources, ensuring that only authorised users and services have access.

Managing user and group permissions

- Adding users to the Docker group
  By default, only the `root` user and members of the `docker` group can run Docker commands.
  Run:
  `sudo usermod -aG docker $USER`
  `newgrp docker`
- Running containers as non-root users
  Run Container with a Specific User:
  `docker run --user 1000:1000 ubuntu whoami`

Best practices for Docker access control

Least Privilege Principle:

- Avoid running containers as root.
- Use `--read-only` for immutable containers.

Network segmentation:

- Use `--network` to isolate containers.

Audit logging:

- Monitor Docker logs (`journalctl -u docker`).

Image trust:

- Use signed images (`docker trust`).

Avoid `--privileged` mode:

- Use `--cap-add` for specific privileges instead.

Conclusion

Effectively managing access control in Docker is crucial for maintaining a secure containerized environment. By leveraging Unix group permissions, running containers as non-root users, securing the Docker daemon with TLS, and implementing RBAC in enterprise setups, you can ensure that only authorized users and services interact with your Docker resources.

Key Takeaways

> - Docker access is controlled via Unix groups (docker group).
> - Running containers as non-root improves security.
> - TLS secures remote Docker API access.
> - TRBAC (in Docker Enterprise) provides granular permissions.